

AULA 4

Laboratório: CRUD Completo



OBJETIVOS DA AULA



AO FINAL DESSA AULA, VOCÊ VAI SER CAPAZ DE...

- Completar o CRUD com as operações de Update e Delete
- Tratar erros com exceções HTTP apropriadas (em vez de devolver vazio/undefined)
- Reaproveitar um DTO de criação num DTO de atualização parcial
- Consolidar boas práticas de API REST vistas desde a Aula 2



OPERAÇÕES CRUD COMPLETAS



O QUE FALTAVA DESDE A AULA 2

```
PATCH /users/:id → atualiza  
parcialmente  
DELETE /users/:id → remove o usuário
```

```
export class UpdateUserDto  
  extends PartialType(CreateUserDto) {}
```

PartialType pega o CreateUserDto e torna todos os campos opcionais — sem duplicar decorators.

POR QUE PARTIALTYPE

- Criar (POST) exige todos os campos — atualizar (PATCH) não deveria
- Sem PartialType, teríamos que reescrever cada `@IsString()/@IsEmail()` de novo, com `@IsOptional()` em cada um
- PartialType reaproveita as mesmas regras do CreateUserDto, só que todas opcionais
- Vem de `@nestjs/mapped-types` (ou `@nestjs/swagger`, se usarem Swagger)



TRATAMENTO DE ERROS



NOTFOUNDEXCEPTION NA PRÁTICA

```
findById(id: number) {  
  const user = this.users.find(  
    u => u.id === id  
  );  
  if (!user) {  
    throw new NotFoundException(  
      `Usuário ${id} não encontrado`  
    );  
  }  
  return user;  
}
```

HTTPEXCEPTION E SUAS SUBCLASSES

- NotFoundException (404), BadRequestException (400), ConflictException (409)...
- Todas herdam de HttpException — o Nest já sabe converter isso em status + JSON
- Pode ser lançada no Service (não só no Controller) — o Nest propaga do mesmo jeito
- Nunca devolva null/undefined silenciosamente onde o certo é um erro HTTP



BOAS PRÁTICAS PARA APIS REST



CONSOLIDANDO O QUE JÁ VIMOS + O QUE ENTRA HOJE

- Verbo certo pra cada ação: GET lê, POST cria, PATCH atualiza parcialmente, DELETE remove
- Status code certo: 200 (ok), 201 (criado), 204 (removido sem corpo), 404 (não encontrado), 400 (dado inválido)
- Exceções específicas (NotFoundException, BadRequestException) em vez de try/catch genérico espalhado
- Nunca acessar o "banco" direto no controller — a regra desde a Aula 1 continua valendo com CRUD completo

